

Internet Programming

CSE341

Lab 7

Servers & Nodejs

Agenda

- Nodejs
- What are servers
- Express

Recap

- Last time we saw how we can expose functions running on a machine to the world using REST APIs
- We also saw how to design a REST API of our own
- We saw how the front end can call these APIs using `fetch()`
- This time we will start implementing the backend logic itself

Nodejs – Basics

- So far, we have been running javascript in the browser
- Node allows us to run javascript on the machine. The same way we run python, C++, Java programs
- To get started, install node
- Make sure that running `node --version` doesn't return with an error

Nodejs – Basics

- Node runs the V8 JavaScript engine. This is the engine that runs in the google chrome browser
- So, it essentially takes the engine out of the browser and runs it locally

Nodejs – Basics

So, what's the difference between Node and the browser

- Both the browser and Node.js use JavaScript as their programming language
- In the browser, most of the time what you are doing is interacting with the DOM using **document** and **window** APIs. Those do not exist in Node.js
- In Node, you have APIs to interact with the file system

Nodejs – Basics

- Let's take an example of a node program that interacts with the file system

```
const fs = require("fs");

console.log("1. Starting to read file...");

const data = fs.readFileSync("example.txt", "utf8");

console.log("2. File contents:");
console.log(data);

console.log("3. Done!");
```


Nodejs – Basics

```
$ node fs_example.js  
1. Starting to read file...  
2. File contents:  
Hello this is node js  
3. Done!
```

Nodejs – Concurrency

- Many programming languages that we have studied before (Java, Python, ...) support multi-threading -> you can simply create a new thread in your program and assign it a task to achieve concurrency
- Unlike these languages, node is single threaded
- At a first glance, this may sound disastrous!
- How can we achieve concurrency??

Nodejs – Concurrency

- Before we go into why node can achieve concurrency, let's first think about some cases where we actually need it
- So that's a question for you: **When would we need concurrency?**

Nodejs – Concurrency

When would we need concurrency?

- If we are running a server, we would want to handle multiple requests at the same time
- If we are doing an I/O operation like reading from the disk or sending a network request, we don't want our entire program to stop and wait for this operation to return
- If we are working with a timer, we don't want our entire program to stop and wait for the time to finish
- If there is a long running background process like compressing a file or sending an email

Nodejs – Concurrency

When would we need concurrency?

We need concurrency whenever we want multiple tasks to *progress* at the same time, especially when some tasks involve slow I/O

A non-concurrent system would freeze whenever it waits for something

Nodejs – Concurrency

> Well, if concurrency is that important, then node is a terrible choice since it is single threaded

No.

Nodejs has a different way to achieve concurrency despite being single threaded

Nodejs – Concurrency

- Allow me to introduce you to the **Event Loop**
- This is the heart of Node's concurrency
- When Node reaches a blocking call (a call that requires waiting), Node does **not** execute it in the main thread.
- It delegates it to the system where the call is executed by another component called “libuv”, and while waiting, the main thread continues executing other code

Nodejs – Concurrency

- When libuv finishes a task, it sends the result back to Node
- The event loop then “resumes” the code after the blocking call

Nodejs – Concurrency

- Let's revisit our file system example

```
const data = fs.readFileSync("example.txt", "utf8");
```

- This is a synchronous call to the read operation
- Meaning that the entire program stops and waits/blocks for the disk I/O to finish before it resumes execution
- Let's make async so that the main thread can execute other code while doing I/O

Nodejs – Concurrency

```
const fs = require("fs");

console.log("1. Starting to read file...");

fs.readFile("example.txt", "utf8", (err, data) => {
  if (err) {
    return console.log("Error:", err);
  }

  console.log("2. File contents:");
  console.log(data);
});

console.log("3. Done!");
```

- Here we gave it a callback to be executed on the output of the async call
- Can you guess the output of the program?

Nodejs – Concurrency

- We can achieve the same thing using async/await to avoid callback hell

```
const fs = require("fs/promises");

async function run() {
  try {
    const data = await fs.readFile("example.txt", "utf8");
    console.log("File contents:", data);
  } catch (err) {
    console.log("Error:", err);
  }
}

run();
```

Nodejs – Concurrency

In summary:

- Node achieves concurrency because your JavaScript is single-threaded but the **I/O system under it is multi-threaded**.
- JS delegates slow operations to libuv threads
- The event loop keeps your program responsive and allows thousands of concurrent requests

What are servers

- Servers are computers that run the backend logic of your application
- They expose their functions to the network
- They sit waiting for network requests and execute the code according to the incoming request
- They return the requested data to the client that sent the request

What are servers

- We can create servers in any programming language
- We will be using javascript and node today for our server
- We will be using ExpressJS: a framework built for node that abstracts and simplifies the process of creating servers

Express

- First we need to install express

```
npm init -y
```

```
npm install express
```

Express

We can create endpoints in express this way:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello from Express!");
});

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Here we created our app object

Then created a get endpoint with path “/”

We provide a callback that will be called when we receive a GET request on the path “/”

We finally tell our app to listen for requests on port 3000

Express

- The call back is called with 2 parameters:
 - Req: an object containing data about the incoming request
 - Res: an object that we can use to make our response

Express

- Here's how we can access path parameters of a request like the id

```
app.get("/users/:id", (req, res) => {  
  res.send(`User ID: ${req.params.id}`);  
});
```

- Using `res.send()` sends text as the response for the request
- We can use `res.json(some_object)` to convert a javascript object to json and send it in the response

Express

- We can add middleware to our express server
- Middleware is a function that runs automatically before your logic
- This middleware for example:

```
app.use(express.json());  
  
app.post("/users", (req, res) => {  
  console.log(req.body); // parsed JSON  
  res.send("Received data");  
});
```

- Parses the json payload of the request into a javascript object

Express

- Let's try to implement the [REST API](#) that we designed last time for the mini-twitter project
- For now, we can use javascripts arrays to store the data

Example for the POST /posts endpoint

```
const express = require("express");
const app = express();
app.use(express.json());

const posts = []

app.post("/posts", (req, res) => {
  // receive a JSON payload like {"title": "...", "content": "..."}
  const post = {
    title: req.body["title"],
    content: req.body["content"]
  }
  console.log(`Adding this new post ${JSON.stringify(post)}`)
  posts.push(post)
  console.log(`Posts: ${JSON.stringify(posts)}`)
  res.status(201)
  res.end()
})

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Task

- Implement the remaining endpoints in the [REST API](#) using javascript arrays and objects for storage